

Sistema Distribuído de Monitoramento e Análise de Eventos Urbanos no Rio de Janeiro

Milton Salgado, Pedro Saito, Eduardo Bensabat, e Gabriel Guimarães

Instituto de Computação – Universidade Federal do Rio de Janeiro (UFRJ)
Rio de Janeiro – RJ – Brasil

Abstract. *This work presents a distributed system for monitoring urban events in Rio de Janeiro, built on a 3-node MongoDB Replica Set deployed with Docker. A data pipeline collects incidents from public Brazilian sources (Fogo Cruzado, INPE, INMET), normalizes them into a canonical GeoJSON-based document schema and completes them with synthetic data up to 100,000 events. Experiments show batch insertion above 16,000 records/s, indexed queries answered in milliseconds at the 100k scale, and consistent operation during the failure of a secondary node, which rejoined the replica set in about 5 seconds.*

Resumo. *Este trabalho apresenta um sistema distribuído de monitoramento de eventos urbanos no Rio de Janeiro, construído sobre um MongoDB Replica Set de 3 nós implantado com Docker. Um pipeline coleta ocorrências de fontes públicas brasileiras (Fogo Cruzado, INPE, INMET), normaliza-as em um esquema canônico de documentos baseado em GeoJSON e as completa com dados sintéticos até 100.000 eventos. Os experimentos mostram inserção em lote acima de 16.000 registros/s, consultas indexadas respondidas em milissegundos na escala de 100 mil documentos e operação consistente durante a falha de um nó secundário, reintegrado em cerca de 5 segundos.*

1. Introdução

Cidades de grande porte geram continuamente ocorrências urbanas, tais como tiroteios, incêndios, alagamentos, quedas de energia, interdições de vias, que são reportadas por cidadãos, sensores e órgãos públicos. Uma plataforma municipal de monitoramento precisa absorver esse fluxo ao mesmo tempo que lida com escalabilidade, suporte a consultas analíticas e geoespaciais e disponibilidade diante de falhas eventuais nos servidores. Esse cenário favorece bancos NoSQL distribuídos, que trocam o esquema rígido e as garantias transacionais dos bancos relacionais por flexibilidade de esquema e maior suporte à replicação e escalabilidade horizontal [Özsu e Valduriez 2020; Sadalage e Fowler 2012].

Este trabalho implementa esse cenário para o município do Rio de Janeiro usando a **Opção A** do enunciado: o banco de documentos **MongoDB**, implantado como Replica Set de 3 nós em Docker. Diferentemente de uma carga puramente sintética, o sistema parte de dados reais das seguintes fontes públicas brasileiras: Ocorrências de violência armada do Instituto Fogo Cruzado [Instituto Fogo Cruzado 2026], focos de queimada do INPE [INPE 2026] e precipitação do INMET [INMET 2026] normalizados em um esquema canônico único e complementados por um gerador sintético determinístico até os volumes requeridos. As contribuições são: (i) um pipeline de dados idempotente em três camadas

(bruta, processada, banco); (ii) a modelagem de documentos com localização GeoJSON e índices adequados às consultas obrigatórias; (iii) um ambiente distribuído reprodutível com replicação e tolerância a falhas; (iv) uma API REST (FastAPI) com dashboard web (React) para exploração dos eventos; e (v) uma avaliação experimental de inserção, consulta e falha de nó nos três volumes de dados. O código está disponível publicamente no GitHub¹.

2. Fundamentação Teórica

Bancos de documentos. Em um banco orientado a documentos, a unidade de armazenamento é um documento semiestruturado (JSON/BSON) autocontido, agrupado em coleções sem esquema fixo [Sadalage e Fowler 2012]. Essa representação não permite junções para entidades agregadas, pois todo evento urbano carrega consigo localização, reportante e proveniência e permite evoluir o esquema sem migrações.

Replicação e tolerância a falhas. Um **Replica Set** do MongoDB é um conjunto de processos mongod que mantêm cópias do mesmo conjunto de dados [MongoDB, Inc. 2024]. Um nó **primário** recebe todas as escritas e as registra no *oplog*, que os **secundários** aplicam assincronamente. Se o primário falha, os nós restantes elegem novo primário por um protocolo de consenso baseado em Raft, desde que haja maioria (quórum): com 3 nós, o sistema tolera a perda de 1. À luz do teorema CAP [Brewer 2000; Gilbert e Lynch 2002], essa configuração prioriza consistência sobre disponibilidade durante partições: escritas exigem contato com o primário, e o *write concern majority* (padrão) só confirma a escrita após replicação para a maioria dos nós.

Índices e consultas geoespaciais. O MongoDB organiza índices secundários em árvores B, e oferece o índice **2dsphere** para geometrias GeoJSON [Butler et al. 2016], suportando operadores como *\$near* e *\$geoWithin* sobre coordenadas esféricas. Consultas analíticas são expressas no *aggregation pipeline* (estágios *\$match*, *\$group*, *\$sort*), executadas no servidor com uso dos índices disponíveis.

3. Modelagem dos Dados

Todo evento, real ou sintético, é normalizado para um único esquema canônico de documento, armazenado na coleção *eventos*:

```
{
  "idEvento": "EVT000001",
  "tipo": "Incêndio",
  "descricao": "Foco de queimada detectado por satélite",
  "dataHora": "2025-06-10T18:30:00Z",
  "gravidade": 4,
  "status": "Aberto",
  "bairro": "Centro", "cidade": "Rio de Janeiro",
  "estado": "RJ", "pais": "Brasil",
  "localizacao": { "type": "Point", "coordinates": [-43.1729, -22.9068] },
  "reportante": { "tipo": "Fonte pública", "identificador": "INPE" },
  "origem": { "fonte": "inpe_bdqueimadas", "idOriginal": "...",
    "arquivoRaw": "data/raw/..." },
  "metadados": { "categoriaOriginal": "...", "extraidoEm": "..." }
}
```

O esquema estende a estrutura mínima do enunciado com proveniência (origem) e metadados específicos por fonte (vítimas, potência radiativa do fogo, precipitação), aproveitando a flexibilidade do modelo de documentos. Os invariantes são: *localizacao* é um Point GeoJSON com ordem [*longitude*, *latitude*] [Butler et al. 2016], requisito

¹<https://github.com/saitoi/trabalho-final-bd2>

do índice 2dsphere; dataHora é ISO 8601 em UTC; gravidade é inteiro de 1 a 5; campos desconhecidos recebem null, nunca valores inventados; e registros sem coordenadas válidas são rejeitados com o motivo registrado. Os tipos de evento seguem uma taxonomia consolidada de 12 categorias (Tiroteio, Incêndio, Alagamento, Chuva intensa, Risco hidrológico, Risco geotécnico, Problema urbano, Transporte, Energia, Vazamento de água, Interdição de via, Outro).

Fontes reais e mapeamento. Cada fonte pública é mapeada para a taxonomia com regras próprias de gravidade: ocorrências do **Fogo Cruzado** viram Tiroteio (gravidade 5 com mortos, 4 com feridos, 3 caso contrário); focos do **INPE BDQueimadas** viram Incêndio (gravidade em função da potência radiativa frp); e estações do **INMET** geram Chuva intensa ou Risco hidrológico conforme limiares de precipitação horária (≥ 25 mm e ≥ 50 mm). A normalização filtra os registros para o município do Rio de Janeiro: de 12.185.229 registros brutos processados, 12.085.229 foram rejeitados como fora do escopo geográfico — decisão registrada em `events_rejected.jsonl`. Como os dados reais do município não alcançam a carga alvo, um gerador sintético determinístico (semente 42), facultado pelo enunciado, completa o restante com eventos distribuídos pelos bairros cariocas e pelas 12 categorias. O corte de 100.000 eventos resulta em 28.490 eventos do Fogo Cruzado, 5.193 do INMET, 1.968 do INPE e 64.349 sintéticos — 100% no Brasil, com os tipos mais frequentes sendo Tiroteio (28.490), Outro (23.517) e Risco hidrológico (14.902).

Coleções e índices. O banco bd2 mantém três coleções: eventos (dados normalizados), fontes_raw_manifest (metadados dos arquivos brutos processados) e experimentos (resultados de benchmark). A Tabela 1 lista os sete índices criados em eventos, um por padrão de acesso obrigatório, mais um índice composto de proveniência que garante idempotência da carga (deduplicação por fonte e identificador original).

Tabela 1. Índices da coleção eventos e consulta que cada um atende.

Índice	Uso
localizacao: 2dsphere	consulta geográfica por raio (\$near)
tipo: 1	consulta por tipo e estatística por tipo
dataHora: 1	consulta por período e evolução temporal
gravidade: 1	consulta por gravidade mínima
bairro: 1	estatística por bairro
cidade: 1	filtros territoriais da API
origem.fonte, origem.idOriginal	deduplicação / idempotência da carga

4. Arquitetura Implementada

A arquitetura separa estritamente três camadas de dados: data/raw/ guarda os arquivos originais das fontes, nunca modificados; data/processed/ recebe todas as transformações; e o MongoDB é carregado exclusivamente a partir dos recortes de benchmark. O fluxo é implementado por scripts Python independentes, idempotentes e parametrizados por CLI, orquestrados por um script único (pipeline.sh):

1. **Extração** (extract_raw_all.py): baixa os dados das APIs públicas (Fogo Cruzado, INPE, INMET, IBGE, OSM) e armazena resultados em ./data/raw;
2. **Inspeção** (inspect_raw_data.py): inventaria a camada bruta sem modificá-la;
3. **Normalização** (normalize_events.py): converte cada fonte ao esquema canônico, aplica o filtro geográfico e o complemento sintético;

4. **Benchmark** (`build_benchmark_datasets.py`): gera recortes determinísticos de 1.000, 50.000 e 100.000 eventos (semente 42);
5. **Índices e carga** (`create_indexes.py`, `load_mongo.py`): criam os índices e inserem os dados em lotes de 1.000 documentos com `insert_many(ordered: false)`;
6. **Consultas e experimentos** (`run_queries.py`, `run_experiments.py`): Executam as consultas obrigatórias e os três testes de desempenho, registrando métricas em JSONL.

Ambiente distribuído. O Docker Compose define o Replica Set `rs0` com três nós MongoDB 7.0 — `mongo1` (porta 27017, prioridade 2, primário preferencial), `mongo2` (27018) e `mongo3` (27019) — cada um com volume próprio e *healthcheck* via `mongosh ping`. Um contêiner auxiliar executa `rs.initiate()` automaticamente na primeira subida, tornando o ambiente reproduzível com um único comando. Scripts e API conectam-se com URI que lista os três nós e `replicaSet=rs0`, de modo que o *driver* descobre a topologia e redireciona operações após eleições.

API e interface web. Uma API REST em FastAPI expõe as funcionalidades obrigatórias: inserção (`POST /events`), consultas por tipo, período, raio geoespacial e gravidade, estatísticas agregadas (por tipo, bairro, dia, gravidade e território) e o estado do Replica Set (`GET /nodes/status`, via `replSetGetStatus`). Um frontend React (Vite, Tailwind, MapLibre, Recharts) oferece mapa interativo com filtros e busca por raio (Figura 1), dashboard analítico, tabela paginada com cadastro de eventos e filtros por metadados, página de benchmarks e monitor dos nós em tempo real.

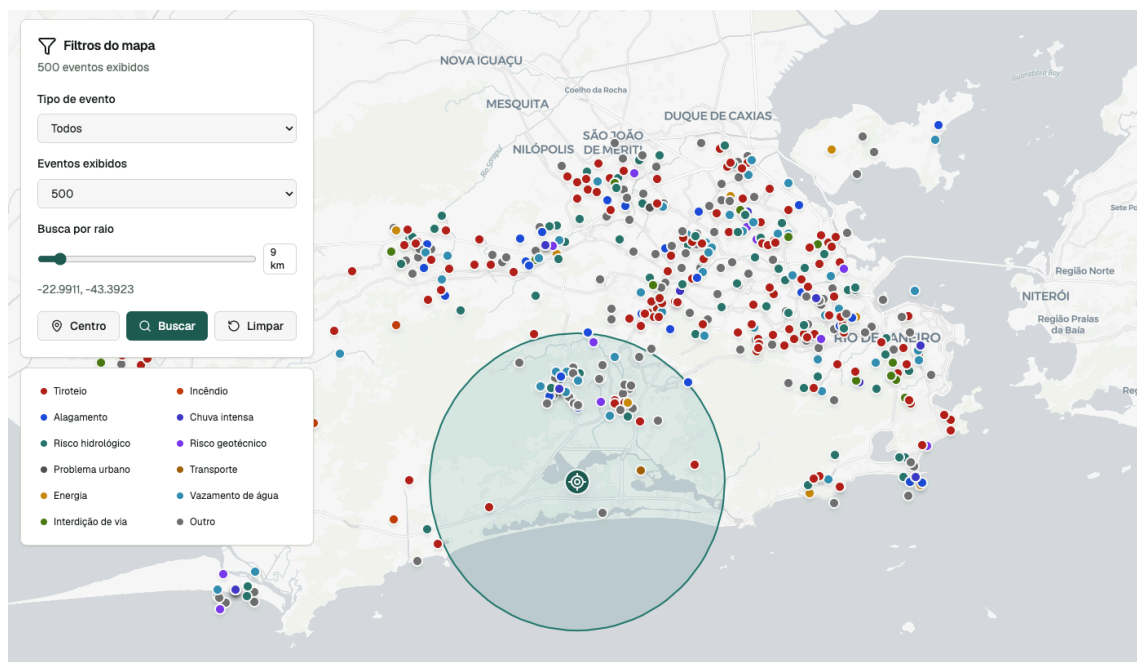


Figura 1. Interface web: mapa dos eventos no Rio de Janeiro, com filtro por tipo e busca geoespacial por raio.

5. Configuração

Os experimentos foram executados em um único host físico, com os três nós do Replica Set emulados como contêineres Docker (Tabela 2). O pipeline usa Python 3.12 com PyMongo 4.10, gerenciado pelo uv; as inserções usam lotes de 1.000 documentos e as configurações padrão de consistência do MongoDB 7.0 (*write concern majority*, leituras

no primário). Todos os processos aleatórios usam semente fixa (42), tornando datasets e experimentos reproduzíveis.

Tabela 2. Configurações do ambiente computacional utilizado.

Componente	Configuração
CPU	Apple M5, 10 núcleos (4 desempenho + 6 eficiência)
RAM	16 GB
Disco	SSD NVMe, 995 GB
SO	macOS 26.5.2, arm64
Banco	MongoDB 7.0 (3 contêineres Docker, Replica Set rs0)
Software	Python 3.12, PyMongo 4.10, Docker Compose

6. Resultados Experimentais

6.1. Teste 1: Inserção

A Tabela 3 apresenta o tempo de inserção em lote para os três volumes, com os sete índices já criados. O tempo cresce de forma aproximadamente linear entre 1 mil e 50 mil registros e superlinear no maior volume, refletindo o custo de manutenção dos índices (em especial o 2dsphere) à medida que a coleção cresce. Em carga isolada, o volume de 100 mil registros foi inserido em 6,2 s (~ 16.050 registros/s), evidenciando que a replicação para os secundários não limita a vazão de escrita nessa escala.

Tabela 3. Teste 1 - tempo de inserção em lote (lotes de 1.000 documentos, ordered: false).

Volume	Tempo (s)	Registros/s
1.000	0,104	9.640
50.000	3,958	12.632
100.000	13,944	7.172

6.2. Teste 2: Consultas

A Tabela 4 mostra o tempo de resposta das três consultas obrigatórias nos três volumes. Todas usam índices: a consulta por tipo usa tipo: 1, a de período usa dataHora: 1 e a geográfica usa o 2dsphere com \$near (raio de 5 km a partir do Centro do Rio). Mesmo no maior volume, todas respondem em menos de 100 ms; o crescimento do tempo é sublinear em relação ao volume, comportamento esperado de buscas indexadas.

Tabela 4. Teste 2 - tempo de resposta (em segundos) por consulta e volume.

Consulta	1.000	50.000	100.000
Por tipo (tipo = "Incêndio")	0,0014	0,0048	0,0247
Por período (ano de 2025)	0,0009	0,0012	0,0028
Geográfica (raio 5 km, Centro)	0,0040	0,0299	0,0920

Sobre o volume de 100 mil documentos, as demais consultas obrigatórias também respondem em frações de segundo: gravidade ≥ 3 em 0,59 s (a mais custosa, por retornar dezenas de milhares de documentos), estatística por tipo (agregação \$group) em 0,14 s, por bairro em 0,19 s e evolução temporal por dia em 0,09 s.

6.3. Teste 3: Falha de Nó

O teste de tolerância a falhas seguiu o procedimento do enunciado: com os 100 mil eventos carregados, executou-se a agregação de contagem por tipo, desligou-se o nó secundário mongo2 (docker stop), repetiu-se a consulta e religou-se o nó. A Tabela 5 resume o resultado: o sistema continuou plenamente operacional com 2 dos 3 nós, a consulta retornou exatamente as mesmas contagens por tipo (verificação automática de consistência) com acréscimo de apenas 37 ms, e o nó religado reintegrou-se ao Replica Set em cerca de 5 s, ressincronizando via *oplog* sem intervenção manual. O estado dos nós antes, durante e após a falha pode ser acompanhado ao vivo na página de monitoramento da interface web (Figura 2), que também documenta o roteiro do teste.

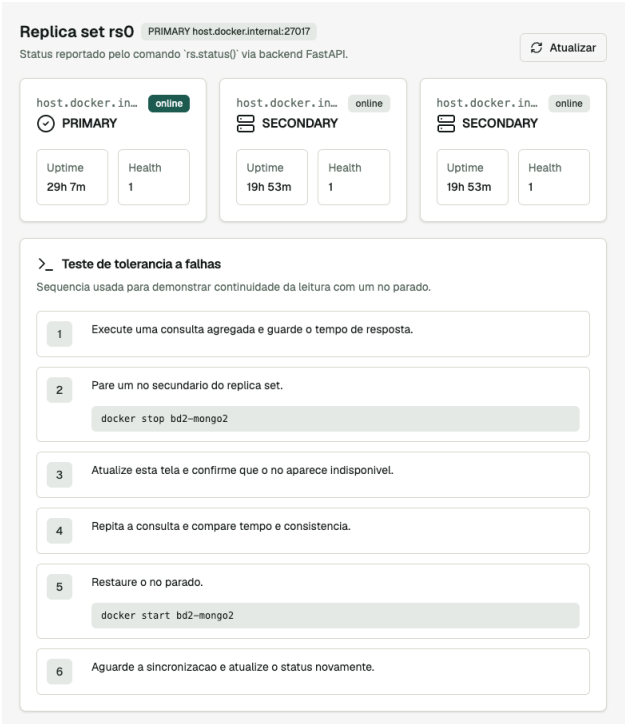


Figura 2. Página de monitoramento do Replica Set rs0 (via `replSetGetStatus`), com o roteiro do teste de tolerância a falhas.

Tabela 5. Teste 3 — falha e recuperação de um nó secundário (agregação por tipo sobre 100.000 documentos).

Métrica	Valor
Consulta com 3 nós ativos	0,231 s
Consulta com mongo2 desligado	0,268 s
Resultados idênticos antes/depois	sim
Reintegração do nó ao religar	5,01 s

7. Discussão

Os experimentos confirmam as propriedades esperadas da arquitetura. Nas **consultas**, os índices mantêm a latência em milissegundos mesmo com o centuplicar do volume: a consulta por período, altamente seletiva, cresce apenas 3× entre 1 mil e 100 mil documentos, enquanto a geográfica — a mais cara, por avaliar distâncias esféricas e retornar ~ 1.800 documentos ordenados por proximidade — permanece abaixo de 0,1

s. Já o tempo das consultas correlaciona-se mais com o tamanho do resultado do que com o tamanho da coleção, como ilustra a consulta por gravidade (0,59 s para ~ 63 mil documentos).

Na **inserção**, a vazão cai no maior volume (7,2 mil reg/s contra 12,6 mil no volume médio), efeito do custo crescente de manutenção dos sete índices; a mesma carga com pipeline de carga dedicado atingiu 16 mil reg/s, indicando que há espaço para otimização (por exemplo, criar índices após a carga inicial).

Na **tolerância a falhas**, a perda de um secundário foi praticamente invisível para a aplicação — 16% de acréscimo em uma consulta de 0,23 s —, pois o quórum de 2/3 preserva o primário e as leituras padrão. O custo dessa disponibilidade aparece nos limites do arranjo: a falha do **primário** implicaria alguns segundos de indisponibilidade de escrita até a eleição, e a perda de dois nós tornaria o conjunto somente leitura, comportamento coerente com a escolha de consistência do MongoDB no espectro CAP [Gilbert e Lynch 2002].

As principais limitações do estudo são: os três nós compartilham o mesmo host físico, de modo que a rede entre eles é local e os tempos absolutos são otimistas em relação a uma implantação geo-distribuída; 64% dos eventos são sintéticos, ainda que geograficamente verossímeis; e não foi avaliado *sharding*, apenas replicação.

8. Conclusão

Este trabalho projetou, implementou e avaliou um sistema distribuído de monitoramento de eventos urbanos para o Rio de Janeiro sobre um MongoDB Replica Set de 3 nós, cumprindo todos os requisitos do enunciado: modelagem NoSQL com esquema canônico e índices geoespaciais, geração dos três volumes de dados (parcialmente a partir de fontes públicas reais), as seis funcionalidades de consulta, ambiente distribuído com replicação demonstrada e os três experimentos com análise. Os resultados mostram que a solução atende consultas analíticas e geoespaciais em milissegundos na escala de 100 mil eventos e permanece disponível e consistente sob falha de um nó. Como trabalhos futuros, destacam-se: avaliar *sharding* por bairro ou tipo para escala horizontal de escrita; explorar *read preference* nos secundários para distribuir carga de leitura; incorporar mais fontes reais (CEMADEN, Portal 1746) reduzindo a fração sintética; e medir o tempo de eleição sob falha do nó primário.

Referências

- Brewer, E. A. (2000). Towards Robust Distributed Systems (Keynote). In *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC)*, p. 7.
- Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S., e Schaub, T. (2016). The GeoJSON Format. <https://www.rfc-editor.org/rfc/rfc7946>
- Gilbert, S., e Lynch, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *ACM SIGACT News*, 33(2), pp. 51–59.
- INMET (2026). Dados Históricos Anuais do Instituto Nacional de Meteorologia. <https://portal.inmet.gov.br/dadoshistoricos>
- INPE (2026). BDQueimadas: Banco de Dados de Queimadas. <https://terrabrasilis.dpi.inpe.br/queimadas/bdqueimadas/>

Instituto Fogo Cruzado (2026). API do Fogo Cruzado: dados abertos sobre violência armada. <https://api.fogocruzado.org.br/>

MongoDB, Inc. (2024). MongoDB Manual 7.0: Replication. <https://www.mongodb.com/docs/manual/replication/>

Özsu, M. T., e Valduriez, P. (2020). Principles of Distributed Database Systems. Springer.

Sadalage, P. J., e Fowler, M. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.